

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Esempi scientifici con i cicli

Lo schema di un calcolo iterativo

Con `if` e i cicli (lezioni 14 e 15) possiamo affrontare problemi veri. Hanno tutti la stessa struttura.

- **Stato:** le variabili che descrivono il sistema a ogni passo (una somma, un intervallo, posizione e velocità…).
- **Inizializzazione:** lo stato di partenza.
- **Aggiornamento:** la regola che porta dal passo n al passo $n + 1$.
- **Arresto:** quando fermarsi (numero di passi, precisione raggiunta, condizione fisica).

Problema	Stato	Arresto
Fattoriale	il prodotto parziale	n passi
Bisezione	l'intervallo $[a, b]$	$b - a <$ tolleranza
Numeri primi	il candidato e i divisori	primo divisore trovato
Fibonacci	gli ultimi due termini	n passi
Caduta di un corpo	posizione e velocità	tempo finale

Per ogni esempio: **test** su un caso noto, e attenzione ai limiti dei tipi (lezioni 3, 4 e 10).

Il fattoriale

$n! = 1 \cdot 2 \cdot 3 \cdots n$: un **accumulatore** che moltiplica. Proviamolo con tre tipi diversi.

```c
#include <stdio.h>

int main() {
    int    f_int  = 1;
    long   f_long = 1;
    double f_dbl  = 1.0;
    int n;

    for (n = 1; n <= 22; n++) {
        f_int  *= n;
        f_long *= n;
        f_dbl  *= n;
        printf("%2d %12d %20ld %.6e\n",
               n, f_int, f_long, f_dbl);
    }
    return 0;
}
```

- L'accumulatore di un prodotto parte da `1`, non da `0`!
- $0! = 1$ per definizione: con $n = 0$ il ciclo non viene eseguito e il risultato è già corretto.
- Il fattoriale cresce **molto** in fretta: $n! \approx \sqrt{2\pi n}\,(n/e)^n$ (formula di Stirling).
- Quale tipo regge più a lungo?

Fattoriale e overflow

```
$ ./fattoriale
...
11     39916800              39916800 3.991680e+07
12    479001600             479001600 4.790016e+08
13   1932053504            6227020800 6.227021e+09
14   1278945280           87178291200 8.717829e+10
...
19    109641728   121645100408832000 1.216451e+17
20  -2102132736  2432902008176640000 2.432902e+18
21  -1195114496 -4249290049419214848 5.109094e+19
22   -522715136 -1250660718674968576 1.124001e+21
```

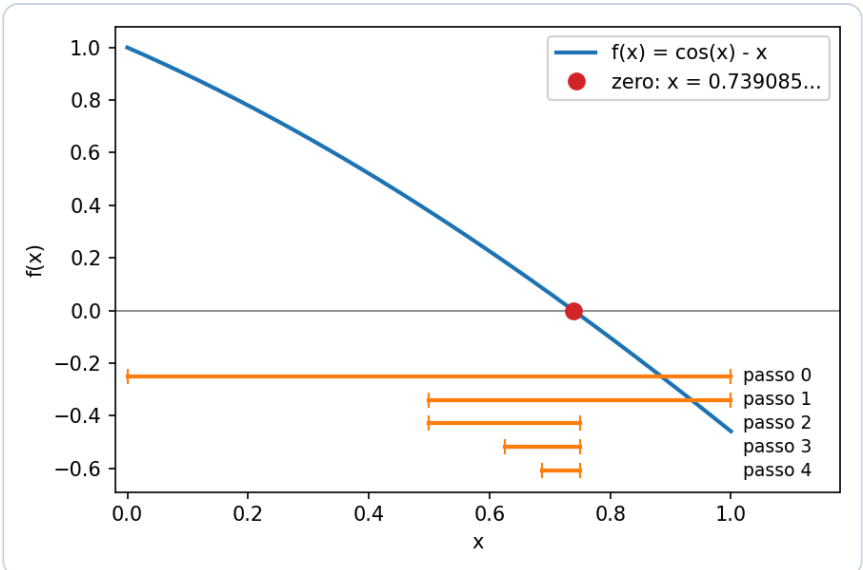
L'overflow di un intero con segno è un comportamento **indefinito**: i valori sbagliati mostrati sono quelli tipici di un PC, ma non sono garantiti.

- `int` (32 bit, max $\approx 2.1 \cdot 10^9$): corretto fino a $12!$; $13! = 6227020800$ non ci sta.
- `long` (64 bit, max $\approx 9.2 \cdot 10^{18}$): corretto fino a $20!$.
- Nessun errore e nessun warning: il risultato è semplicemente **sbagliato** (lezione 3), a volte persino negativo.
- `double` : arriva fino a $170! \approx 7.3 \cdot 10^{306}$, poi `inf` ; ma ha solo ~16 cifre significative (lezione 4).

Ricerca degli zeri: la bisezione

Problema: risolvere $\cos x = x$, cioè trovare lo zero di $f(x) = \cos x - x$.

- $f(0) = 1 > 0$ e $f(1) = \cos 1 - 1 < 0$: f è continua, quindi ha uno zero in $[0, 1]$.
- Si prende il punto medio $c = (a + b)/2$.
- Se $f(a)$ e $f(c)$ hanno segno **opposto**, lo zero è in $[a, c]$; altrimenti è in $[c, b]$.
- Si ripete: a ogni passo l'intervallo si **dimezza**.
- Dopo n passi la larghezza è $(b - a)/2^n$: per una tolleranza di 10^{-10} servono $\log_2 10^{10} \approx 34$ passi.



La bisezione in C

```c
#include <stdio.h>
#include <math.h>
int main() {
    const double TOL = 1e-10;
    double a = 0.0, b = 1.0;          // f(a) > 0, f(b) < 0
    double fa = cos(a) - a;
    int n = 0;
    while (b - a > TOL) {
        double c  = 0.5 * (a + b);
        double fc = cos(c) - c;
        if (fa * fc <= 0) {           // lo zero e' in [a, c]
            b = c;
        } else {                      // lo zero e' in [c, b]
            a = c;
            fa = fc;
        }
        n++;
    }
    double x = 0.5 * (a + b);
    printf("x = %.10f, cos(x) = %.10f\n", x, cos(x));
    printf("%d iterazioni\n", n);
    return 0;
}
```

```
$ ./bisezione
x = 0.7390851332, cos(x) = 0.7390851332
34 iterazioni
```

- **Test:** $\cos x = x$ alle 10 cifre stampate ✓; 34 iterazioni, come previsto.
- `fa` viene ricalcolato solo quando `a` cambia: una valutazione di f per passo.
- La bisezione è **lenta** ma **sicura**: se all'inizio c'è un cambio di segno, converge sempre.
- Vale per qualunque funzione continua: basta cambiare le righe con `cos`.

Numeri primi

```c
#include <stdio.h>

int main() {
    int n, d, quanti = 0;
    for (n = 2; n <= 100; n++) {
        int primo = 1;                  // ipotesi: n e' primo
        for (d = 2; primo && d * d <= n; d++) {
            if (n % d == 0) {
                primo = 0;              // trovato un divisore
            }
        }
        if (primo) {
            quanti++;
            printf("%4d", n);
            if (quanti % 10 == 0) {
                printf("\n");
            }
        }
    }
    printf("\nPrimi fino a 100: %d\n", quanti);
    return 0;
}
```

```
$ ./primi
   2   3   5   7  11  13  17  19  23  29
  31  37  41  43  47  53  59  61  67  71
  73  79  83  89  97
Primi fino a 100: 25
```

- Due cicli **annidati**: per ogni candidato `n`, si provano i divisori `d`.
- Basta arrivare a $d \le \sqrt{n}$: se $n = d \cdot q$, uno dei due fattori è $\le \sqrt{n}$. Si scrive `d * d <= n`, senza `sqrt`.
- La condizione `primo && ...` fa uscire dal ciclo interno al primo divisore trovato.
- **Test:** i primi minori di 100 sono 25 ✓

Fibonacci e il rapporto aureo

$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$. Il rapporto F_n / F_{n-1} tende a $\varphi = \frac{1+\sqrt{5}}{2}$.

```c
#include <stdio.h>
#include <math.h>

int main() {
    const double PHI = (1 + sqrt(5)) / 2;   // rapporto aureo
    long f_prec = 0, f = 1;                 // F(0), F(1)
    int n;

    for (n = 2; n <= 40; n++) {
        long f_nuovo = f + f_prec;
        f_prec = f;
        f = f_nuovo;
        if (n % 5 == 0) {
            double r = (double) f / f_prec;
            printf("%2d %10ld %.12f %9.2e\n", n, f, r, r - PHI);
        }
    }
    return 0;
}
```

```
$ ./fibonacci
 5          5 1.666666666667  4.86e-02
10         55 1.617647058824 -3.87e-04
15        610 1.618037135279  3.15e-06
20       6765 1.618033963167 -2.56e-08
25      75025 1.618033988958  2.08e-10
30     832040 1.618033988748 -1.69e-12
35    9227465 1.618033988750  1.38e-14
40  102334155 1.618033988750 -2.22e-16
```

- Bastano **due** variabili per lo stato; la terza è temporanea.
- L'errore cambia segno e si riduce di ≈ 120 volte ogni 5 passi, fino all'epsilon di macchina.
- Con `long`, F_{93} va già in overflow.

Simulare la caduta di un corpo

- Le leggi del moto sono equazioni **differenziali**:
  $\dfrac{dy}{dt} = v, \qquad \dfrac{dv}{dt} = -g.$
- Il calcolatore non sa fare limiti: dividiamo il tempo in **passi** di durata Δt.
- Per Δt piccolo: $y(t + \Delta t) \approx y(t) + v(t)\,\Delta t$.
- **Metodo di Eulero:**

$$y_{n+1} = y_n + v_n\,\Delta t, \qquad v_{n+1} = v_n - g\,\Delta t$$

- Qui la soluzione esatta è nota, $y(t) = h_0 - \frac{1}{2}gt^2$: ottimo **test** per il metodo.
- Lo stesso schema funziona quando la soluzione esatta **non** si conosce: attrito dell'aria, molle non lineari, pianeti…
- Lo **stato** è la coppia (y, v); l'**aggiornamento** è il passo di Eulero; l'**arresto** è il tempo finale.

Il passo usa la velocità **vecchia** v_n: per questo calcoliamo `y` **prima** di aggiornare `v`.

La caduta in C

```c
#include <stdio.h>

int main() {
    const double g  = 9.81;      // [m/s^2]
    const double h0 = 20.0;      // altezza iniziale [m]
    const double T  = 2.0;       // durata [s]
    const int    N  = 20;        // numero di passi
    double dt = T / N;           // passo temporale [s]
    double y = h0, v = 0.0;      // [m], [m/s]
    int i;

    printf("# t[s]   y_eulero[m]  y_esatta[m]\n");
    for (i = 0; i <= N; i++) {
        double t = i * dt;
        printf("%6.3f %12.5f %12.5f\n", t, y, h0 - 0.5 * g * t * t);
        y = y + v * dt;          // passo di Eulero
        v = v - g * dt;
    }
    return 0;
}
```

```
$ ./caduta > caduta.dat
$ cat caduta.dat
# t[s]   y_eulero[m]  y_esatta[m]
 0.000     20.00000     20.00000
 0.100     20.00000     19.95095
 0.200     19.90190     19.80380
 0.300     19.70570     19.55855
...
 1.900      3.22490      2.29295
 2.000      1.36100      0.38000
```

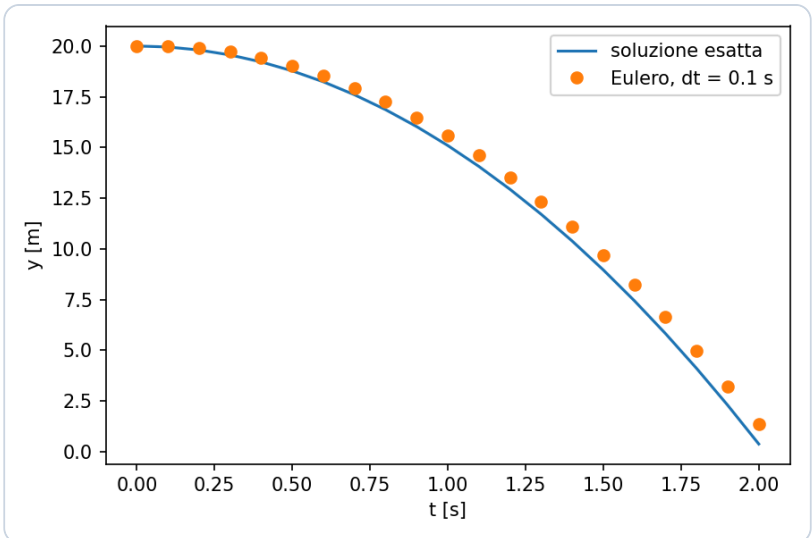
- Contatore **intero** e `t = i * dt` (lezione 15).
- Output in colonne, con intestazione `#`: pronto per `np.loadtxt` (lezione 12).
- Eulero resta **indietro** rispetto alla caduta vera.

Il grafico con Python

```python
import numpy as np
import matplotlib.pyplot as plt

t, y_eu, y_es = np.loadtxt('caduta.dat',
                           unpack=True)

plt.plot(t, y_es, '-', label='soluzione esatta')
plt.plot(t, y_eu, 'o', label='Eulero, dt = 0.1 s')
plt.xlabel('t [s]')
plt.ylabel('y [m]')
plt.legend()
plt.savefig('caduta_eulero.png')
```



Il C **calcola**, Python **disegna**: ognuno fa quello che sa fare meglio.

Quanto deve essere piccolo dt?

Cambiamo `N` e ricompiliamo: altezza stimata da Eulero e suo errore a $t = 2$ s.

`N`	Δt [s]	y_{Eulero} [m]	Errore [m]
20	0.1	1.36100	0.98100
200	0.01	0.47810	0.09810
2000	0.001	0.38981	0.00981
20000	0.0001	0.38098	0.00098

Valore esatto: $y(2\ \text{s}) = 0.38000$ m. Ultima riga di `./caduta` per ogni `N`.

- L'errore è **proporzionale** a Δt: dieci volte più passi, un errore dieci volte più piccolo. Eulero è un metodo del **primo ordine**.
- Qui si può dimostrare: $y_{\text{Eulero}} - y_{\text{esatta}} = \frac{1}{2} g\, t\, \Delta t$, cioè $9.81 \cdot \Delta t$ a $t = 2$ s ✓
- Ma più passi significa più **tempo di calcolo** e più errori di **arrotondamento** accumulati.
- Esistono metodi più precisi (Eulero-Cromer, Verlet, Runge-Kutta): li incontrerete più avanti.